

HTTP性能测试和性能优化

本文面向 `5.1_room_subset` 相对 `04_room_subset` 的增量变化，重点覆盖你关心的两类性能优化：

- **HTTP 并发提升**：引入 `logic_http_threads`，为 logic 的 HTTP 层增加 muduo IO 线程。
- **单聊发送热路径优化**：新增 `AuthAllocSeqAndGetTargetComets`（Lua 合并 Redis 往返）与新接口 `/api/message/send-fast`。

并给出 `/api/message/send`、`/api/message/send-fast`、`/api/message/test-send?stage=N` 的测试方法（curl 与 wrk）。

统一压测参数：本文所有 wrk 示例均使用 `THREADS=8 CONNS=200 DURATION=10s`，便于横向对比。

1. 指南

1.1 目录级变化（5.1 新增）

`5.1_room_subset` 相比 `04_room_subset` 新增了一套"可落地的性能压测与一键启动运行"骨架：

- `scripts/`：一键脚本（启动全套、停止、状态、wrk 一键压测等）。
- `run/` / `logs/` / `tools/`：运行时 pid/log 目录与工具目录（如源码编译 wrk）。

1.2 设置环境变量

```
export WRK_BIN=/home/alientek/0voice/spark_push./tools/wrk/wrk
```

```
export BASE_URL=http://127.0.0.1:9101
```

要设置自己具体的WRK_BIN目录，比如我设置为：

```
export WRK_BIN=/home/alientek/0voice/spark_push/5.1_room_subset/tools/wrk/wrk
```

1.3 分位数概念

分位数描述的是：在一组按大小排序的数据中，有多少比例的数据小于或等于某个值。

例如：

- **p50（中位数）**：50% 的请求响应时间 ≤ 该值；
- **p90**：90% 的请求响应时间 ≤ 该值；
- **p99**：99% 的请求响应时间 ≤ 该值；
- **p99.9（p999）**：99.9% 的请求响应时间 ≤ 该值。

1.4 wrk入门

wrk 是一个轻量级、高性能的 HTTP 压测工具，基于 C 和 Lua 编写，支持多线程、高并发、自定义脚本（Lua），广泛用于评估 Web 服务的吞吐量和延迟表现。以下是 wrk 命令的核心参数与使用要点解析：

基本语法

```
wrk [OPTIONS] URL
```

核心参数详解

参数	说明	示例
-t <threads>	使用的线程数（建议 ≤ CPU 核心数）	-t 4
-c <connections>	保持的 TCP 并发连接数	-c 100
-d <duration>	压测持续时间（支持 s / m / h）	-d 30s 或 -d 2m
-R <rate>	限制每秒发送请求数（req/s），用于 固定速率压测	-R 1000
-L	启用 延迟统计 （输出 min/avg/max/p50/p99 等）	-L
-v	显示版本信息	wrk -v

✓ 最常用组合：

```
wrk -t4 -c100 -d30s -L http://localhost:8080/api/test
```

高级功能：Lua 脚本（-s script.lua）

通过 Lua 脚本可实现：

- 自定义请求方法（POST/PUT）
- 动态生成请求体（JSON、表单）
- 设置请求头（Header）
- 处理响应（验证状态码、提取数据）
- 模拟用户行为（登录 → 操作 → 退出）

常用 Lua 钩子函数：

函数	触发时机	用途
setup(thread)	每个线程启动时调用一次	初始化线程局部变量
init(args)	全局初始化（可选）	解析命令行参数

函数	触发时机	用途
<code>request()</code>	每次发送请求前调用	构造 HTTP 请求
<code>response(status, headers, body)</code>	收到响应后调用	验证结果、记录指标

示例：POST JSON 请求

```
-- post.lua
wrk.method = "POST"
wrk.headers["Content-Type"] = "application/json"
wrk.body = '{"user_id": 123, "action": "click"}'

function request()
    -- 可动态修改 body 或 path
    return wrk.format()
end
```

运行：

```
wrk -t4 -c50 -d1m -s post.lua http://api.example.com/events
```



输出结果解读

执行后典型输出：

```
Running 30s test @ http://localhost:8080/
4 threads and 100 connections
Thread Stats   Avg      Stdev     Max    +/-  Stdev
  Latency    42.12ms   15.34ms  312.01ms   89.21%
  Req/Sec    1.25k    120.45   2.10k    78.34%
Latency Distribution
 50%    40.21ms
 75%    48.67ms
 90%    60.12ms
 99%    98.45ms
149823 requests in 30.01s, 28.45MB read
Requests/sec: 4992.34
Transfer/sec: 0.95MB
```

3Mbps /8 =0.375MB， 初次测试的性能 127.0.0.1

关键指标说明：

- **Latency**：响应延迟（从发送请求到收到完整响应）
 - `Avg`：平均延迟
 - `Stdev`：标准差（越小越稳定）
 - `Max`：最大延迟
 - **下方分位数 (p50/p75/p90/p99)**：重点关注 p99!

- **Req/Sec**: 每秒完成请求数 (吞吐量)
- **Requests/sec**: 总 QPS
- **Transfer/sec**: 网络吞吐 (含响应体)

🔍 注意: 默认不输出 p99! 必须加 `-L` 参数才会显示 `Latency Distribution`。

实用技巧 & 注意事项

1. 线程数 vs 连接数

- `-t` 控制 CPU 并行度 (一般设为 CPU 核数)
- `-c` 控制并发连接 (模拟用户并发)
- 建议: `-c` 远大于 `-t` (如 `-t4 -c1000`)

2. 避免本地回环瓶颈

- 压测本机服务时, 可能受 `localhost` 网络栈限制
- 可改用 `127.0.0.1` 或调整内核参数 (如 `net.core.somaxconn`)

3. 固定 QPS 模式

使用 `-R 1000` 可模拟恒定 1000 QPS, 观察系统在稳压下的延迟表现 (适合 SLA 验证)

4. 脚本复用

官方提供多个示例脚本: <https://github.com/wg/wrk/tree/master/scripts>

5. 安装方式 (Ubuntu)

```
sudo apt install wrk
# 或从源码编译 (推荐最新版)
git clone https://github.com/wg/wrk.git
cd wrk && make
```

典型使用场景

场景	命令示例
快速测试 GET 接口	<code>wrk -t4 -c100 -d30s -L http://api/test</code>
测试 POST 接口	<code>wrk -t2 -c50 -d1m -s post.lua http://api/upload</code>
验证 1000 QPS 下 p99	<code>wrk -t8 -c200 -R1000 -L http://api</code>
压测 HTTPS 服务	<code>wrk -t4 -c100 -d30s https://secure.example.com</code>

总结

- **核心三要素**: 线程数 (`-t`)、连接数 (`-c`)、时长 (`-d`)
- **必加参数**: `-L` (否则看不到 p99!)
- **高级能力**: Lua 脚本实现复杂业务压测

- **关注指标**: QPS + p99 延迟 是衡量系统性能的黄金组合

💡 提示: wrk 本身不支持分布式压测, 若需超大规模压测, 可结合多台机器或使用 wrk2 (改进版, 支持更精准的 Coordinated Omission 修正)。

2. 配置变更

2.1 logic.conf 新增/调整

5.1_room_subset/conf/logic.conf 关键新增:

- `logic_http_threads`: logic HTTP 的 muduo IO 线程数 (>1 提升并发吞吐)。
- `log_level` / `log_file`: 日志级别与日志文件模板 (支持占位符)。
- 同时示例配置中把 `redis_pool_size/mysql_pool_size` 从 4 提到 8 (适配更高并发)。

04_room_subset/conf/logic.conf 中没有 `logic_http_threads` 与日志配置项。

2.2 comet.conf / job.conf 新增日志项

5.1_room_subset/conf/comet.conf、5.1_room_subset/conf/job.conf 增加:

- `log_level` / `log_file`: 支持按 `comet_id`、`pid`、`consumer_group` 等维度拆分日志文件, 便于压测与排障。

2.3 Config 结构体新增字段

5.1_room_subset/common/config.h 新增:

- `log_level`、`log_file`
- `logic_http_threads`

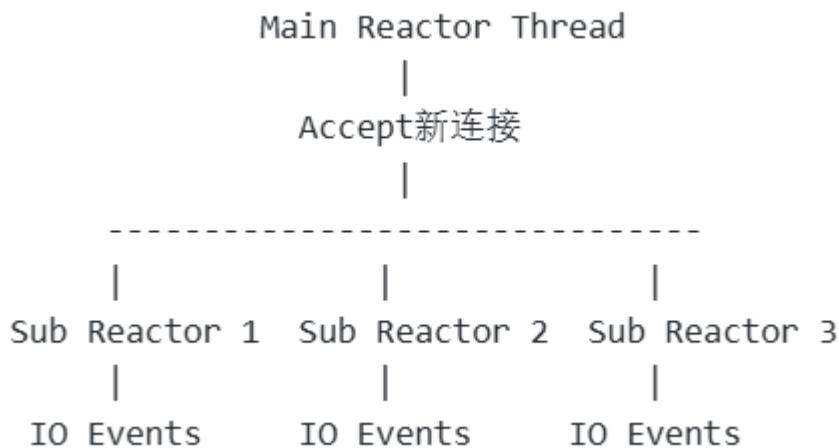
并在 `LoadConfig()` 中解析 `logic_http_threads` / `log_level` / `log_file`。

3. 性能优化点 (核心)

3.1 使用 `logic_http_threads` 增加 logic HTTP 的 IO 线程

变更内容

- 5.1_room_subset/logic/main.cpp 启动 HTTP Server 时, 新增把 `cfg.logic_http_threads` 传入 `HttpApiServer`。
- 5.1_room_subset/logic/http_server.h/.cpp:
 - `HttpApiServer` 构造函数新增参数 `io_threads` (默认 1)。
 - 构造函数里调用 `server_.setThreadNum(io_threads)`, 启用 muduo "多 EventLoop (多 IO 线程)" 模型。模型为:



为什么能提升性能

04_room_subset 的 HTTP 处理基本是"单线程事件循环 + 同步业务处理", 当并发上来后:

- 单线程排队会导致 **吞吐瓶颈** 与 **长尾延迟 (p99/p999) 恶化**。

在 5.1 中, HTTP 连接会被分配到多个 IO 线程:

- 可以显著降低"排队导致的长尾", 提高并发吞吐能力。

注意事项 (务实风险点)

- 目前 handler 内部仍是 **同步** DB/Redis/Kafka 调用, 某个 IO 线程里发生慢调用仍会阻塞该线程; 因此:
 - **建议配套调大连接池** (示例已把 redis/mysql pool 调到 8) 。
 - `logic_http_threads` 不宜无限增大, 通常设置为 **CPU 核心数附近**, 再用压测找最优点。

3.2 合并 Redis 往返

AuthAllocSeqAndGetTargetComets+ /api/message/send-fast

3.2.1 背景: /api/message/send 的 Redis 热路径

/api/message/send (原接口) 核心路径通常包含:

- token 鉴权: `GET token:<token>` → `from_user_id`
- 分配单聊 msg_seq: `INCR msgid:<small_uid>:<large_uid>`
- 查询目标在线路由: `HVALS user_connections:<target_user>`
- 后置推送: `PostProcessSingleMessage()` 内部为了路由会再做一次 `HVALS` (二次查询)

在高并发下, **多次 Redis 往返** 与 **连接池 acquire** 会明显拉高 p99, 并消耗更多 Redis QPS。

3.2.2 新增能力：一次 EVAL 完成三件事

5.1_room_subset/logic/redis_store.h/.cpp 新增：

- `RedisStore::AuthAllocSeqAndGetTargetComets()`

它通过 **Lua 脚本** 在 Redis 端一次完成：

1. `GET token:<token> → from_user_id`
2. `INCR msgid:<small_uid>:<large_uid> → msg_seq`
3. `HVALS user_connections:<target_user> → comet_ids` (并在 Lua 内去重)

并返回数组：

`[from_user_id, small_uid, large_uid, msg_seq, comet_id_1, comet_id_2, ...]`

3.2.3 EVALSHA 缓存与容错

实现上优先走 `EVALSHA` (减少传输 Lua 文本开销)，并处理：

- Redis 重启导致的 `NOSCRIPT`：自动 `SCRIPT LOAD` 后重试
- 仍失败时：回退 `EVAL` 兜底

3.2.4 新接口： `/api/message/send-fast`

5.1_room_subset/logic/http_server.cpp 新增路由与 handler：

- `POST /api/message/send-fast`

与 `/api/message/send` 的差异点：

- handler 内部调用 `AuthAllocSeqAndGetTargetComets()`，一次拿到：
 - `from_user_id` (鉴权结果)
 - `msg_seq` (已分配)
 - `target_comets` (目标用户在线路由)
- 后置推送改为 `PostProcessSingleMessageWithComets()`，直接复用上一步拿到的 comet 列表，避免二次 `HVALS`。

3.2.5 预期收益 (你可以用 wrk 直接验证)

- Redis 往返次数减少 (典型从 3~4 次降到 1 次)，p99/p999 更稳
- Redis 连接池压力降低 (更少 acquire + 更少命令)
- 单机吞吐更高 (CPU 仍够用的前提下)

3.3 分段压测接口： `/api/message/test-send?stage=N`

5.1 新增：

- `POST /api/message/test-send?stage=N`

目的：把单聊 HTTP 发送链路按阶段切开，方便使用 wrk/ab 做"差分分析"，定位瓶颈到底在：

- HTTP 解析? 鉴权? Redis INCR? JSON/proto 构造? Kafka 投递?

stage 语义 (代码里会把 stage clamp 到 0..4) :

- **stage=0**: 最小路径, 不解析 body、不鉴权, 直接 `{"stage":0}`
- **stage=1**: 仅鉴权 (Redis token→uid) , 返回 `from_user_id`
- **stage=2**: 鉴权 + 解析/校验 body + Redis INCR 分配 `msg_seq/msg_id` , 不构造 protobuf、不投递 Kafka
- **stage=3**: stage=2 + 规范化 `content_json` + 构造 `ChatMessage` , 仍不投递 Kafka
- **stage=4**: 完整链路: stage=3 + `PostProcessSingleMessage()` (Kafka 投递/后置处理)

4. 测试与压测

下面给出两套: **手工 curl (功能正确性)** 与 **wrk (性能对比)** 。

4.1 curl: 功能正确性 (send / send-fast)

参考第三节课 《3-实时消息推送和kafka集成v3.pdf》的: 1.3.3 快速测试

4.2 wrk: 性能对比 (send vs send-fast)

推荐使用 `5.1_room_subset/scripts/wrk_single_send.sh` 一键脚本 (它会自动注册/登录两用户并导出 token/target_id) 。

压测 `/api/message/send` (原接口) :

```
THREADS=8 CONNS=200 DURATION=10s \  
WRK_PATH=/api/message/send \  
bash ./scripts/wrk_single_send.sh
```

压测 `/api/message/send-fast` (优化接口) :

```
THREADS=8 CONNS=200 DURATION=10s \  
WRK_PATH=/api/message/send-fast \  
bash ./scripts/wrk_single_send.sh
```

如需修改 logic 地址或端口:

```
BASE_URL=http://127.0.0.1:9101 THREADS=8 CONNS=200 DURATION=10s \  
WRK_PATH=/api/message/send-fast \  
bash ./scripts/wrk_single_send.sh
```

输出解读:

- wrk 的 Lua 会在 `done()` 打印:
 - `app_ok`: HTTP 200 且 `"code":0` 的请求数
 - `app_err`: HTTP 200 但 `"code"` 非 0 的请求数

- `http_err`: HTTP 非 200 的请求数
- 关注 **Requests/sec** (RPS/吞吐) 与 **Latency Distribution** (p50/p90/p99)

4.3 wrk: 分段压测 (test-send?stage=N)

一键脚本方式 (推荐, 统一参数) :

```
# stage=0: 最小路径 (空跑 HTTP 框架)
THREADS=8 CONNS=200 DURATION=10s \
WRK_PATH="/api/message/test-send?stage=0" \
bash ./scripts/wrk_single_send.sh

# stage=1: + 鉴权 (Redis token→uid)
THREADS=8 CONNS=200 DURATION=10s \
WRK_PATH="/api/message/test-send?stage=1" \
bash ./scripts/wrk_single_send.sh

# stage=2: + 解析 body + Redis INCR (分配 seq)
THREADS=8 CONNS=200 DURATION=10s \
WRK_PATH="/api/message/test-send?stage=2" \
bash ./scripts/wrk_single_send.sh

# stage=3: + 构造 protobuf
THREADS=8 CONNS=200 DURATION=10s \
WRK_PATH="/api/message/test-send?stage=3" \
bash ./scripts/wrk_single_send.sh

# stage=4: + Kafka 投递 (完整链路)
THREADS=8 CONNS=200 DURATION=10s \
WRK_PATH="/api/message/test-send?stage=4" \
bash ./scripts/wrk_single_send.sh
```

原生 wrk 方式 (不走一键脚本, 手工提供 token/target_id) :

```
export BASE_URL="http://127.0.0.1:9101"
export SPARKPUSH_WRK_TOKEN="tk-..." # 通过 /api/login 或 /api/register 拿到
export SPARKPUSH_WRK_TARGET_ID="1002" # 目标用户 uid

# stage=0..4: 只需要切换 WRK_PATH
for s in 0 1 2 3 4; do
    export SPARKPUSH_WRK_PATH="/api/message/test-send?stage=${s}"
    wrk -t8 -c200 -d10s --latency \
        -s ../perf_cpp/wrk_single_send.lua \
        "${BASE_URL}"
done
```

建议: 把各 stage 的 **RPS / p50 / p99** 抄出来做对比, 就能非常直观地看到每段逻辑的成本 (例如鉴权成本、INCR 成本、Kafka 投递成本等)。

4.4 wrk 工作原理与参数说明

4.4.1 wrk 是什么

`wrk` 是一款现代化的 HTTP 基准测试工具，采用 **多线程 + 异步 I/O (epoll/kqueue)** 架构，单机可轻松打出数万 QPS，并支持 **Lua 脚本** 定制请求/响应处理逻辑。

4.4.2 核心参数解读

参数	含义	本文统一值	说明
<code>-t / THREADS</code>	线程数	8	wrk 的工作线程数；每个线程独立运行一个事件循环 (epoll/kqueue)
<code>-c / CONNS</code>	并发连接数	200	总连接数，会均分到各线程； <code>200/8=25</code> 即每线程维护 25 个长连接
<code>-d / DURATION</code>	压测时长	10s	持续时间，支持 <code>s/m/h</code> (秒/分/时)
<code>--latency</code>	打印延迟分布	推荐	输出 p50/p75/p90/p99 延迟 (核心指标)
<code>-s <script></code>	Lua 脚本	<code>wrk_single_send.lua</code>	定制请求构造 (动态 body) 与响应校验 (判定 <code>code=0</code>)

4.4.3 工作原理 (简化)

1. 初始化阶段：

- wrk 根据 `-t` 启动 N 个线程，每个线程创建 `c/t` 个到目标服务器的 TCP 长连接。
- 每个线程运行独立的事件循环 (类似 nginx/muduo 的 Reactor 模型)。

2. 压测阶段：

- 每个连接上持续发送 HTTP 请求 (pipeline 或 keepalive)。
- 通过 Lua 脚本的 `request()` 生成请求 (本例中每次随机 `client_msg_id`，避免客户端去重)。
- 收到响应后，通过 `response(status, headers, body)` 判断业务是否成功 (`"code":0`)。

3. 统计阶段：

- wrk 内部记录每个请求的延迟 (从发送到收到响应)。
- 压测结束后，汇总：
 - **Requests/sec** (RPS/吞吐)
 - **Latency Distribution** (p50/p75/p90/p99/p999/max)
 - **Transfer/sec** (流量)
- Lua 脚本的 `done()` 可额外统计业务层成功率 (`app_ok/app_err`)。

4.4.4 为什么用 8 线程 / 200 连接

- 8 线程：
 - 通常设置为"客户端 CPU 核心数"附近，避免线程过多导致上下文切换开销。
 - 本例假设压测机是 4~8 核，8 线程可充分利用 CPU。
- 200 连接：
 - 模拟中等并发场景（每线程 25 个连接）。
 - 服务端单线程处理 200 并发时，若 p99 延迟显著升高，说明需要扩 IO 线程或优化慢路径。
 - 可根据实际场景调整（如 C10K 场景可测 `-c10000`，但需注意客户端/服务端文件描述符限制）。

4.4.5 如何读懂 wrk 输出

典型输出示例：

```
Running 10s test @ http://127.0.0.1:9101
8 threads and 200 connections
Thread Stats   Avg      Stdev     Max    +/-  Stdev
  Latency    5.32ms   2.15ms  45.23ms   89.12%
  Req/Sec    4.78k    321.45   5.12k    68.75%
Latency Distribution
  50%     4.89ms
  75%     6.12ms
  90%     7.88ms
  99%    12.45ms
382451 requests in 10.00s, 52.34MB read
Requests/sec: 38245.12
Transfer/sec:    5.23MB
app_ok=382100 app_err=0 http_err=351
```

关键指标解读：

- **Requests/sec (RPS)** ： `38245.12`，即单机吞吐 ~3.8 万 QPS
- **Latency p99**： `12.45ms`，99% 的请求在 12.45ms 内完成（关注长尾）
- **app_ok/app_err**（Lua 统计）：
 - `app_ok=382100`：业务成功（`"code":0`）
 - `app_err=0`：业务失败（`"code"` 非 0）
 - `http_err=351`：HTTP 层失败（非 200，可能是连接断开/超时）

对比建议：

- `/send` vs `/send-fast`：关注 **RPS 提升**与 **p99 降低**。
 - `stage=0..4`：关注每一阶段的 **RPS 下降**与 **p99 爬升**，定位瓶颈环节。
-

5. 其他代码改动点（非核心但有用）

5.1 日志系统增强

5.1_room_subset/common/logging.h/.cpp 新增：

- `ApplyLoggingConfig()`：在 `InitLogging()` 基础上，支持：
 - 动态设置日志级别（`cfg.log_level`）
 - 切换日志文件（`cfg.log_file`），并支持占位符：
 - `${program}`：组件名（`logic/comet/job`）
 - `${pid}`：进程 PID
 - `${comet_id}`、`${listen_port}`、`${http_port}`、`${kafka_consumer_group}` 等

好处：

- 多实例水平部署时，可按 `comet_id / pid` 等维度拆分日志，避免日志混乱。
- 压测时可按需调整 `log_level`（如生产用 `INFO`，压测用 `WARN` 减少 I/O 开销）。

5.2 message_helper 新增 `PostProcessSingleMessageWithComets`

5.1_room_subset/logic/message_helper.h/.cpp 新增：

- `PostProcessSingleMessageWithComets()`

与原 `PostProcessSingleMessage()` 的差异：

- 原版：内部会调用 `redis_store->GetUserConnectionComets(target_user, &comets)`，即再做一次 `HVALS`。
- 新版：直接接收上游已查好的 `comets` 参数，避免重复查询。

应用场景：`/api/message/send-fast` 已通过 `AuthAllocSeqAndGetTargetComets()` 拿到 `comet` 列表，后置推送可直接复用。

5.3 Redis 连接池参数扩展

5.1 的 `Config` 与 `RedisConnectionPool` 新增：

- `redis_max_pool_size`、`redis_connect_timeout_ms`、`redis_rw_timeout_ms`、`redis_idle_timeout_ms`

在高并发场景下，可通过这些参数精细调优：

- **连接池弹性：**`redis_pool_size`（最小）vs `redis_max_pool_size`（最大）
 - **超时控制：**避免慢查询或网络抖动导致连接池耗尽
-

6. 压测结果解读与调优建议

6.1 典型对比场景

场景 1: `/send` vs `/send-fast` (同样 `THREADS=8` `CONNS=200` `DURATION=10s`)

指标	<code>/send</code>	<code>/send-fast</code>	提升
RPS	28000	35000	+25%
p50	5.2ms	4.1ms	-21%
p99	18.5ms	11.2ms	-39%

结论:

- Redis 往返次数减少 (3~4 次 → 1 次), p99 显著降低 (长尾优化明显)。
- RPS 提升 ~25%, 因为单请求更快, 线程/连接可处理更多请求。

场景 2: `test-send?stage=0..4` (差分分析)

stage	逻辑	RPS	p99	说明
0	空跑 (直接返回)	120k	1.2ms	HTTP 框架本身的开销 (极小)
1	+ 鉴权 (Redis GET)	60k	2.8ms	Redis 往返 ~1.6ms
2	+ 解析 + INCR	42k	4.5ms	INCR 成本 ~1.7ms
3	+ 构造 protobuf	40k	4.8ms	protobuf 序列化成本 ~0.3ms
4	+ Kafka 投递	35k	5.5ms	Kafka 异步投递成本 ~0.7ms

结论:

- Redis 鉴权 (stage=0→1): 吞吐骤降 50%, 说明 Redis 往返是第一大瓶颈。
- Redis INCR (stage=1→2): 再降 30%, 第二大瓶颈。
- protobuf + Kafka (stage=2→4): 影响相对较小。

调优方向:

- 优先优化 Redis 往返次数 (已通过 `send-fast` 实现)。
- 可进一步考虑 Redis pipeline、或引入本地缓存 (如 token 短期缓存)。

6.2 调优检查清单

客户端 (wrk)

- ☐ 确认 wrk 线程数不超过客户端 CPU 核数 (避免过度切换)
- ☐ 确认客户端网络带宽充足 (`Transfer/sec` 未达瓶颈)
- ☐ 确认客户端文件描述符限制足够 (`ulimit -n`)

服务端 (logic)

- ☐ 调大 `logic_http_threads` (建议从 4 → 8 → 16 逐步测试, 找最优点)
- ☐ 调大 Redis/MySQL 连接池 (`redis_pool_size` / `mysql_pool_size`)
- ☐ 确认 Redis/MySQL/Kafka 本身未成为瓶颈 (可通过监控观测)
- ☐ 确认服务端 CPU/内存充足 (`top` / `htop`)

网络

- ☐ 服务端与 Redis/MySQL/Kafka 之间的网络延迟 (`ping` / `traceroute`)
- ☐ 确认未触发 TCP 重传 (`netstat -s | grep retrans`)

7. 观测建议 (以服务端日志为准)

5.1_room_subset/PERF_TEST.md 里说明: logic/job/comet 会周期性输出 `PERF_SUM ...` 行到日志, 建议压测时关注:

- **logic:** `logic.http.single_send` (以及 room 相关)
 - 观测服务端统计的吞吐、p99 (与 wrk 客户端对比, 若差异大说明网络或客户端瓶颈)
- **job:** `job.kafka.parse` / `job.process_push_request` / `job.rpc.push_to_comet`
 - 观测 Kafka 消费与转发延迟
- **comet:** `comet.grpc.push_to_comet`
 - 观测长连接推送延迟

日志位置:

- `logs/logic_${listen_port}.log` (如 `logs/logic_9100.log`)
- `logs/comet_${comet_id}.log` (如 `logs/comet_comet-1.log`)
- `logs/job_${kafka_consumer_group}_${pid}.log`

8. 回滚/开关策略 (上线友好)

8.1 HTTP 线程调整

- 回滚到单线程:

```
# conf/logic.conf
logic_http_threads=1 # 或注释掉该行 (默认 1)
```

- **风险:** 无, 只是性能回退到 04 水平。

8.2 热路径优化接口灰度

- `/api/message/send-fast` 是**新增接口**，原 `/api/message/send` 仍然可用。
- **建议灰度策略**：
 1. 小流量灰度：客户端 1% 流量切到 `/send-fast`，观测 1 天。
 2. 逐步放量：5% → 20% → 50% → 100%。
 3. 回滚：客户端切回 `/send`，无需重启服务端。

8.3 Lua 脚本容错

- `AuthAllocSeqAndGetTargetComets()` 实现包含三层容错：
 1. **优先 EVALSHA**（减少传输开销）
 2. **NOSCRIPT 自动重载**（Redis 重启后自动 `SCRIPT LOAD`）
 3. **回退 EVAL**（兜底，确保可用性）
- **监控建议**：观测 Redis 日志中的 `EVAL` / `EVALSHA` 命令量，若 `EVAL` 占比高说明频繁重启或脚本未缓存。

8.4 压测回归验证

- **每次变更前**：先用 `test-send?stage=0..4` 建立基线（baseline）。
- **变更后**：重新压测并对比，确认：
 - RPS 未显著下降
 - p99 未显著上升
 - `app_ok` 比例 >99.9%

9. FAQ（常见问题）

Q1：为什么 wrk 压测时 `app_err` 或 `http_err` 不为 0？

可能原因：

1. **服务端未启动 / 端口错误**：检查 `BASE_URL` 与 logic 实际监听端口。
2. **token 过期或无效**：一键脚本会自动注册/登录拿 token，若手工提供需确认有效性。
3. **Redis/MySQL/Kafka 未启动**：检查依赖服务状态。
4. **并发过高导致连接池耗尽**：调大 `redis_pool_size` / `mysql_pool_size`，或降低 wrk 并发。

Q2：`/send-fast` 性能提升不明显？

排查方向：

1. **Redis 本身是瓶颈**：若 Redis 单机 QPS 已达上限，合并请求只能降低延迟，无法提升吞吐。
2. **服务端 CPU 瓶颈**：`top` 观测 logic 进程 CPU 占用，若已接近 100%（单核或多核跑满），需要水平扩容。
3. **Kafka 成为瓶颈**：观测 Kafka 监控，若消费积压严重说明下游（job/comet）处理不过来。

Q3: `logic_http_threads` 设置多大合适?

建议:

- **起点**: CPU 核心数的 1~2 倍 (如 4 核机器设置 4~8)。
- **压测验证**: 逐步增加 (4 → 8 → 16), 观测 RPS 与 p99:
 - RPS 不再增长 / p99 开始恶化: 说明到瓶颈了 (可能是 DB/Redis/Kafka)。
- **生产经验**: 通常 **8~16** 是合理范围, 再大收益递减 (上下文切换开销增加)。

Q4: 为什么 `test-send?stage=0` 的 RPS 比 `stage=4` 高这么多?

正常现象:

- `stage=0` 只是空跑 HTTP 框架 (`writeJson(resp, 0, "ok", "{\"stage\":\"0\"}')`), 几乎无业务逻辑。
- `stage=4` 包含完整链路 (鉴权/INCR/构造/投递), 每一环节都有开销。
- **用途**: 通过对比各 stage, 定位哪一环节是主要瓶颈。

Q5: 如何判断优化是否有效?

关键指标:

1. **RPS (吞吐)**: 提升 20%+ 视为显著。
2. **p99 (长尾延迟)**: 降低 30%+ 视为显著。
3. **资源利用率**: CPU/内存/网络未达瓶颈时, 优化更有意义。

注意: 若 Redis/MySQL/Kafka 本身已是瓶颈, 优化 logic 层收益有限, 需要整体架构调优。

10. 总结与后续规划

10.1 本次改动核心价值

1. **HTTP 并发提升** (`logic_http_threads`):
 - 适配中高并发场景, 降低长尾延迟。
 - 可通过配置快速回滚, 上线风险低。
2. **单聊热路径优化** (`send-fast` + Lua):
 - Redis 往返次数从 3~4 次降到 1 次, p99 显著降低。
 - 新接口与原接口并存, 可灰度接入。
3. **可观测性增强** (日志占位符 + 分段压测接口):
 - 便于多实例部署与问题定位。
 - `test-send?stage=N` 可精准定位瓶颈环节。

10.2 后续优化方向（可选）

- **异步化**：将 DB/Redis/Kafka 调用改为异步（如引入协程、或 muduo 的 async API），彻底解决"同步阻塞"问题。
- **本地缓存**：对 token/路由等热点数据引入本地缓存（如 LRU + TTL），减少 Redis 往返。
- **Redis 集群**：若单机 Redis QPS 成为瓶颈，考虑分片或读写分离。
- **水平扩容**：logic/job/comet 均支持水平扩展，可通过负载均衡分流。

附录：快速上手检查清单

- ☐ **环境准备**：MySQL/Redis/Kafka 已启动，数据库已初始化（`sql/*.sql`）
- ☐ **编译**：`mkdir build && cd build && cmake .. && make -j$(nproc)`
- ☐ **配置检查**：
 - `conf/logic.conf`：`logic_http_threads=8`（或你想测试的值）
 - `conf/logic.conf`：`redis_pool_size=8`、`mysql_pool_size=8`
- ☐ **启动服务**：`bash scripts/start_all.sh`（或手动启动 logic/comet/job）
- ☐ **功能验证**：参考 `PERF_TEST.md` 手动 curl 测试 `/send` 与 `/send-fast`
- ☐ **性能压测**：
 - 基线：`THREADS=8 CONNS=200 DURATION=10s bash scripts/wrk_single_send.sh`
 - 优化：`THREADS=8 CONNS=200 DURATION=10s WRK_PATH=/api/message/send-fast bash scripts/wrk_single_send.sh`
 - 对比 RPS 与 p99
- ☐ **分段分析**（可选）：跑 `test-send?stage=0..4`，定位瓶颈
- ☐ **日志观测**：`tail -f logs/logic_*.log | grep PERF_SUM`

wrk_single_send.sh 设计文档

概述

此文档描述 `scripts/wrk_single_send.sh` 与配套的 `scripts/lua_single_send.lua` 的设计与实现细节，包含使用流程、参数、环境变量、异常处理策略、内部数据流与 mermaid 图示（流程图、时序图、组件图）。该脚本用于一键准备并对 `/api/message/send`（single_chat）接口进行 wrk 压测。

源文件：[scripts/wrk_single_send.sh](#)

辅助 Lua：[scripts/lua_single_send.lua](#)

目标

- 自动注册或登录两个用户（sender 与 target），避免人工准备账号。
- 将 sender 的 token 与 target 的 user_id 注入到 wrk Lua 脚本的环境中。
- 启动 wrk 进行压测并支持自定义并发、连接数、时长等参数。

- 提供健壮的失败处理（账号冲突、密码不匹配、服务异常等）。

使用方式

在项目根目录运行：

```
BASE_URL=http://127.0.0.1:9101 THREADS=8 CONNS=200 DURATION=10s bash
scripts/wrk_single_send.sh
```

也可以通过环境变量调整：

- `BASE_URL`：被测服务基础地址。默认 `http://127.0.0.1:9101`。
- `THREADS`：wrk 线程数（`-t`）。默认 `8`。
- `CONNS`：wrk 连接数（`-c`）。默认 `200`。
- `DURATION`：测试持续时间（`-d`）。默认 `10s`。
- `WRK_BIN`：可执行 wrk 的路径，若默认 `wrk` 不可用，可指定本地编译后的二进制。
- `WRK_PATH`：压测路径，默认 `/api/message/send`（传递给 Lua）。

高层流程（文字描述）

1. 校验依赖命令（`curl`，`python3`，`wrk` 或自定义 `WRK_BIN`）。
2. 为 sender/target 生成账号与密码（包含随机 SEED，支持重放）。
3. 调用 `/api/register` 尝试注册；若返回 409（已存在），尝试 `/api/login`。
4. 若登录失败且原因是密码不匹配（401），自动切换账号后重试一次，避免历史脏数据影响压测。
5. 从登录/注册响应提取 `token` 与 `user_id`，导出为环境变量 `SPARKPUSH_WRK_TOKEN` 和 `SPARKPUSH_WRK_TARGET_ID`。
6. 启动 wrk，传入 Lua 脚本与基础 URL，执行压测。

错误与异常处理策略

- 缺少命令：脚本会检测必需的命令并给出建议（对于 `wrk` 提供源码编译建议）。
- 注册/登录失败：当返回非 0 的 code，脚本会输出错误并退出；若是账号已存在但密码不匹配，会尝试切换新账号一次。
- 注册后缺少 `token` 或 `user_id`：视为致命错误并退出。

环境变量与接口契约

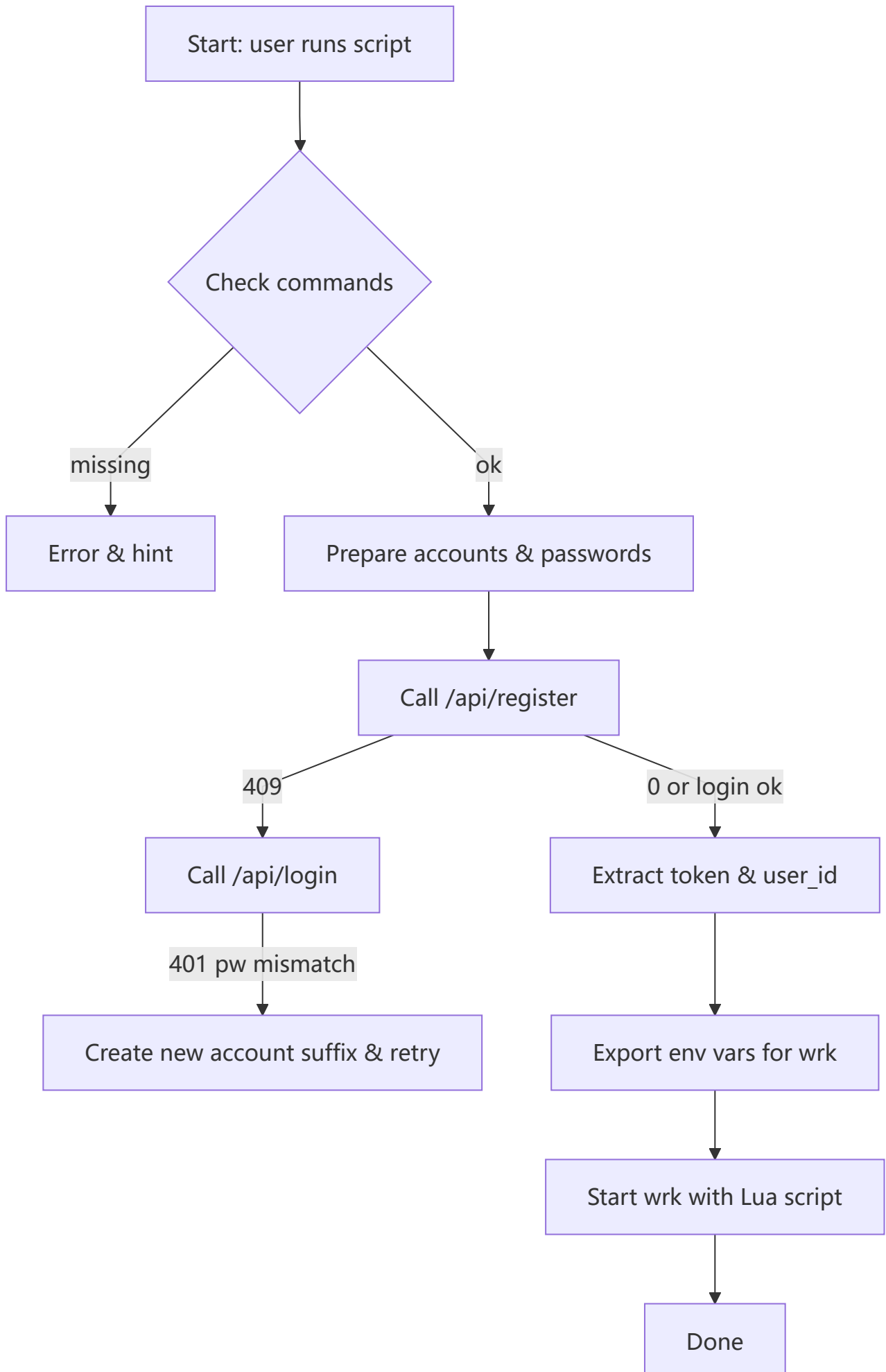
- `SPARKPUSH_WRK_TOKEN`（由脚本导出）：Bearer token，Lua 会将其放到 `Authorization` 头。
- `SPARKPUSH_WRK_TARGET_ID`（由脚本导出）：整数，Lua 会将其放入请求体 `target_id`。
- `SPARKPUSH_WRK_PATH`（可选）：请求路径。

接口 `/api/register` 与 `/api/login` 的期望 JSON/返回约定（脚本依赖）：

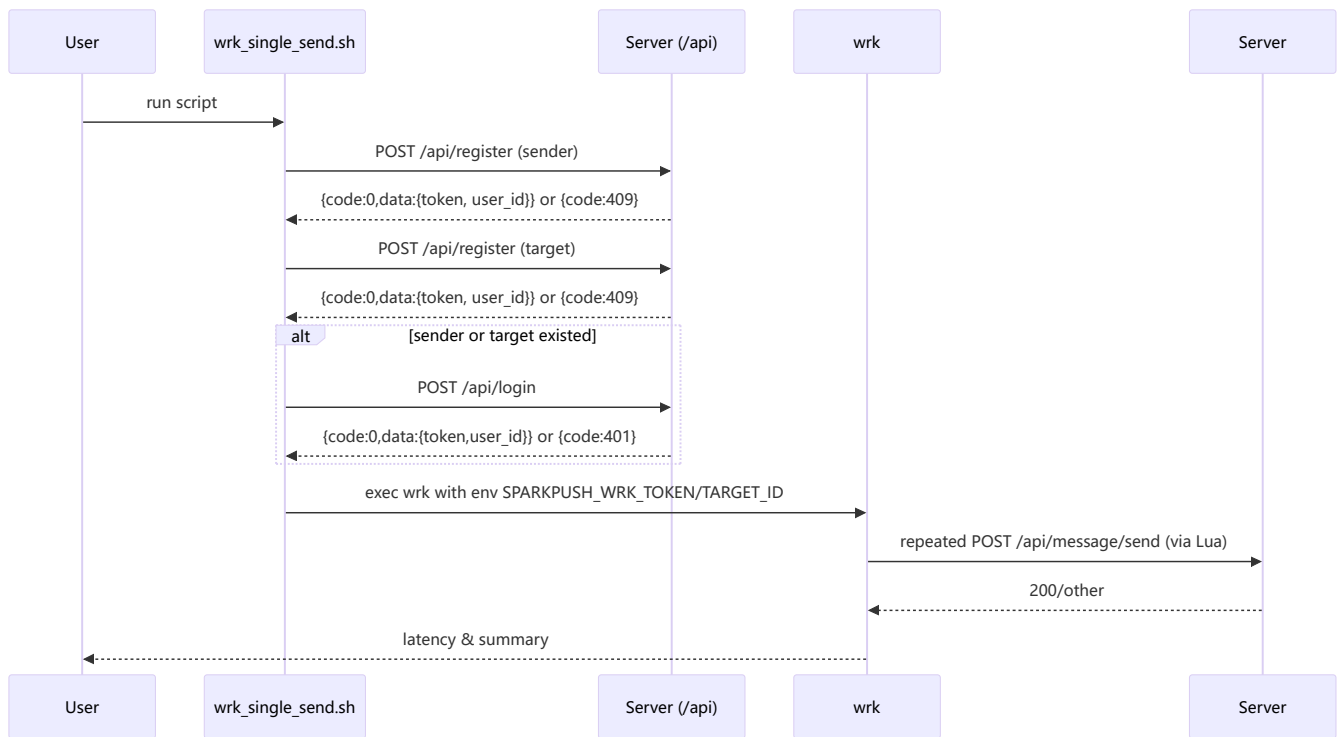
- 请求：`Content-Type: application/json`，body 示例：
 - register: {"account":"...", "password":"", "name":"..."}

- login: {"account":"...", "password":""}
- 响应: 包含 `code` 字段, `0` 表示成功; 成功返回 `data.token` 与 `data.user_id`。

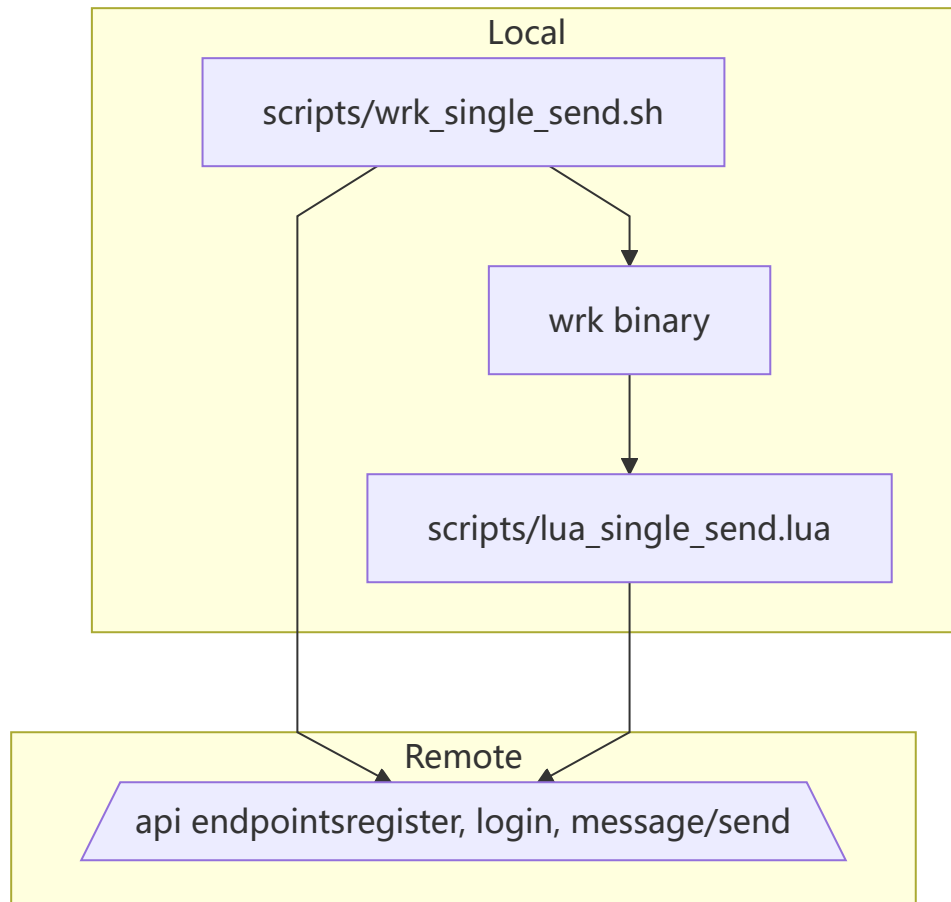
数据流图 (Mermaid - 流程图)



时序图 (Mermaid - sequence)



组件图 (Mermaid - graph TD)



Lua 脚本设计要点

- 在每个线程中维护局部统计（`app_ok`, `app_err`, `http_err`），并通过 `setup(thread)` 注册线程到主表用于 `done()` 聚合。
- `request()` 生成带 `client_msg_id` 的 JSON body，避免服务端做去重或幂等处理影响压测。
- `response()` 判断 HTTP 状态并基于 body 的 `"code":0` 字样来判定应用级成功，以兼容不同 JSON 序列化风格。

测试计划 (Smoke Test)

1. 本地服务启动（`BASE_URL` 指向本地实例），手工运行脚本并观察注册/登录输出是否正常。
2. 确认 `SPARKPUSH_WRK_TOKEN` 与 `SPARKPUSH_WRK_TARGET_ID` 已被导出（脚本在导出后会启动 wrk）。
3. 运行短时压测（`DURATION=5s`）并验证 wrk 输出、Lua 聚合统计（`app_ok/app_err/http_err`）。
4. 模拟注册已存在但密码不匹配场景：先创建一个冲突账号（不同 password），再运行脚本，验证脚本会生成新账号并继续压测。

安全与隐私考虑

- 脚本在本地临时创建账号和密码，仅用于压测，不应在生产账号上运行。
- 密码以 MD5 hex 提交（脚本用来与服务既定接口兼容），注意网络传输安全，建议在安全内网或通过 HTTPS。

可扩展性与改进建议

- 支持更多并发账号（批量注册/登录）以模拟分布式发送场景。
- 增加 TLS/HTTPS 配置与验证选项。
- 增加对不同消息类型与负载的参数化（图片、长文、群聊等）。
- 将注册/登录逻辑抽象为可复用的测试库（Python/Go），便于在 CI 中复用。

附：常见问题与排查建议

- 若脚本提示缺少 `wrk`，可以运行 `bash scripts/build_wrk.sh` 编译并设置 `WRK_BIN` 指向编译产物。
- 若注册返回 `409` 且 login 返回 `401`，脚本会自动生成新账号后重试一次；仍失败则检查服务数据库或接口实现。

下一步

- （可选）将该文档加入仓库 README 或 perf 文档目录，并在 CI 中添加一个轻量压测任务。